HYBRID DEEP LEARNING FRAMEWORK FOR ADAPTIVE MALICIOUS LINK DETECTION AND WEB THREAT INTELLIGENCE

A. Nikhitha[1], K. Sanjana[2], and K. Hemanth Sathyanarayana[3]

Department of Computer Science and Engineering (Cyber Security)
Malla Reddy University, Hyderabad, Telangana, India

[1]2311CS040013@mallareddyuniversity.ac.in
[2]2311CS040083@mallareddyuniversity.ac.in
[3]2311CS040086@mallareddyuniversity.ac.in

ABSTRACT More people use the Internet every day, and attacks such as phishing and malware are growing along with it. These attacks try to steal personal data or install harmful software. Older tools check a link against a blacklist or a set of fixed rules, so they often miss new harmful websites. This paper presents a hybrid deep learning framework that finds harmful links and gives the user useful threat information. It uses Machine Learning, Deep Learning and Agentic AI together. For every URL, the system picks out features such as URL length, domain details, HTTPS use, number of subdomains and suspicious words. A Random Forest model and a neural network then use these features to place the URL into one of four groups: Benign, Phishing, Malware or Defacement. The work is shared among several small agents. One checks the URL, one extracts features, one makes the prediction, one studies the threat and one writes the report. This lets the whole process run without manual steps. At the end, the user gets the result, a confidence score, a risk level and simple safety advice. Tests show that the system finds harmful URLs with high accuracy and few wrong predictions. By joining the three techniques, the framework gives a practical and scalable way to protect users from new online threats.

INDEX TERMS Hybrid Deep Learning, Agentic AI, Malicious Link Detection, Web Threat Intelligence, Phishing Website Detection, Machine Learning, Cybersecurity, URL Classification.

I. INTRODUCTION

The Internet has grown very fast, and so have the attacks on its users. Attackers use phishing pages, malware sites and defaced websites to steal passwords, spread harmful software and take control of devices. Spotting a dangerous link before someone clicks it is now an important task in cybersecurity. It is also a hard one, because attackers keep changing their tricks to get past security tools [1].Most older tools depend on blacklists and hand-written rules. They work well for links that are already known, but a brand-new or slightly changed link goes unnoticed until someone reports it. Blacklists also need constant updating, so new attacks (zero-day attacks) get through in the meantime [2].Machine Learning helps with this problem. A model can learn from many labelled URLs and then judge new ones by their features. Algorithms such as Random Forest, Decision Tree, Support Vector Machine and Logistic Regression have separated safe and harmful sites with good accuracy [3]. Deep Learning goes further. It can find complex patterns in the features on its own, which helps against advanced phishing and malware [4].Agentic AI is a newer idea. Instead of one big program doing everything, several small agents each take one job and work together. For URL checking, one agent can validate the link, another can extract features, another can predict, and others can analyse the threat and write the report. This makes the system more automatic and flexible, and people have to do less by hand [5].In this paper we combine these ideas in one framework. It uses a Random Forest, a neural network and a group of agents to sort URLs into Benign, Phishing, Malware and Defacement. The agents also work out the confidence, judge the risk and prepare a report. Our aim is a real-time tool that is more accurate, gives fewer false alarms and gives users clear advice on what to do.

II. RELATED WORK

Many researchers have worked on harmful URL detection. This section groups the earlier work into machine learning methods, deep learning and hybrid models, agent-based methods and threat intelligence systems. It then explains where our work fits.

A. MACHINE LEARNING-BASED DETECTION

Many studies take lexical and host-based features from URLs and give them to common classifiers such as Random Forest, Decision Tree, SVM and Logistic Regression [2], [3], [6], [7]. Some work has also looked at choosing the best features to improve accuracy and speed [5]. Among these classifiers, Random Forest is often reported as the best, because many trees voting together give stable results and less overfitting. The main weakness is that the features are chosen by hand, so new attack patterns can be missed.

B. DEEP LEARNING AND HYBRID MODELS

Neural networks learn useful patterns straight from the data. Earlier studies have used sequence models and BiLSTM-based designs for URL and phishing detection [9], [14]. Other studies use transformer and language-model detectors [8], [10], [16],

and one study checked how well such detectors resist attacks designed to fool them [13]. Hybrid models mix classic and neural methods to get the best of both. They usually give better accuracy and fewer false positives than a single model. However, most of them stop at classification and do little threat analysis or automatic decision making.

C. AGENTIC AI IN CYBERSECURITY

With Agentic AI, independent agents split a security job and cooperate. Tasks such as validating a URL, extracting features, predicting and reporting can each go to its own agent. This improves scalability and reduces manual work. Not many studies have used it for harmful URL detection yet, so there is room for more research.

D. WEB THREAT INTELLIGENCE

Threat intelligence systems look at signs such as domain details, HTTPS status, IP-based addresses, suspicious words and URL structure to describe how risky a site is [11], [12], [16]. Many of them rely on outside data feeds that must be kept up to date, so they react slowly to newly created phishing and malware pages.

E. WHERE THIS WORK FITS

Most earlier studies improve one detection method on its own. They seldom make several parts work together or automate security tasks. Our framework joins Random Forest, a neural network and an agent-based pipeline in one system. It checks the URL, extracts features, predicts the class, estimates confidence and risk, and writes a report automatically. This supports real-time detection and threat intelligence.

III. PROBLEM STATEMENT AND DESIGN OBJECTIVES

Phishing, malware, defacement and fraud are rising, but popular detection tools use blacklists and basic machine learning, which struggle with sites they have never seen. Many tools also only label the URL. They do not explain the threat, decide automatically or advise the user. So we need an adaptive framework that combines machine learning, deep learning and Agentic AI.

A. PROBLEM STATEMENT

Our study of current systems showed four main problems.

1) Zero-day attacks. Blacklists and signatures only know old threats, so new phishing, malware and fraud links often go undetected.

2) Limited features. Models that use hand-picked lexical features can miss deeper links between URL attributes, so they do worse against advanced threats.

3) Little automation. Most tools stop after classification. People still have to analyse the threat and write the report, which takes time.

4) Scalability and adaptability. Attack styles keep changing, and current systems need frequent retraining and manual updates.

B. DESIGN OBJECTIVES

We designed the framework to meet these objectives.

1) Accurate detection: sort URLs correctly into Benign, Phishing, Malware or Defacement using hybrid AI.

2) Useful features: extract lexical, domain-based and security-related attributes that help both models.

3) Hybrid model: combine Random Forest and a neural network for better accuracy and fewer wrong results.

4) Agent-based automation: let separate agents do validation, feature extraction, prediction, analysis and reporting with little human help.

5) Real-time intelligence: show users the confidence score, threat level, risk analysis and tips for safe browsing.

6) Scalable and adaptive design: keep the system flexible so it can follow new threats and accept better models later.

IV. AGENTIC AI FRAMEWORK ARCHITECTURE

The framework automates the whole detection process using cooperating agents instead of one large module. Six task agents work under an Orchestrator Agent, which controls the order of the steps and passes data between them. The output of each agent becomes the input of the next. This modular design is easier to maintain, extend and adapt, and it gives the user a confidence score, a risk level and recommendations.

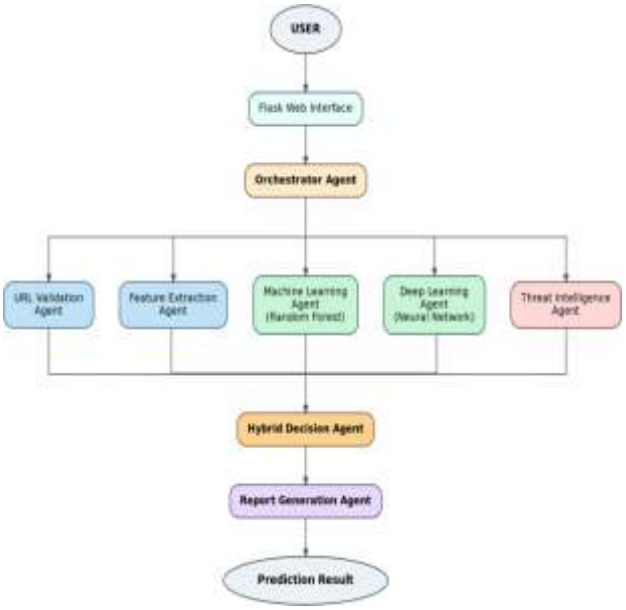


Fig. 1. System Flow Chart

Fig. 1 shows the flow. The user enters a URL in the Flask web interface. The Orchestrator Agent manages the workflow. The URL Validation Agent checks the address and sends it to the Feature Extraction Agent, which gets the lexical, domain-based and security features. The Machine Learning Agent (Random Forest) and the Deep Learning Agent (neural network) then work on the features separately, and the Hybrid Decision Agent combines their results. The Threat Intelligence Agent works out the confidence score, risk score and threat level, and the Report Generation Agent shows the final report with advice on the web page.

A. ORCHESTRATOR AGENT

This agent is the main controller. It takes the URL from the user, passes it to the validation step, tracks progress and makes sure each agent's output reaches the next step in the right order.

B. URL VALIDATION AGENT

This agent checks that the input is a proper URL with a valid domain and a suitable protocol. Wrong or broken entries are rejected. Valid ones go on to feature extraction. This saves computing effort and makes the system more reliable.

C. FEATURE EXTRACTION AGENT

This agent turns a valid URL into a set of numbers called a feature vector. The features include URL and domain length, HTTPS availability, number of dots and subdomains, special characters, use of an IP address and suspicious keywords. Both models use this vector as input.

D. MACHINE LEARNING AGENT

This agent runs a Random Forest on the feature vector. The forest builds many decision trees, and their votes decide which of the four classes the URL belongs to. Using many trees improves accuracy and reduces overfitting.

E. DEEP LEARNING AGENT

This agent sends the same vector through a 1D-CNN + BiLSTM + Attention model. The model can pick up complex, hidden relationships that tree-based models may not capture. Its prediction supports the Random Forest and goes to the Hybrid Decision Agent.

F. HYBRID DECISION AGENT

This agent compares the outputs of the two models along with their confidence values and picks the final class. Using both models gives better accuracy and fewer false positives than using one.

G. THREAT INTELLIGENCE AGENT

This agent studies the final prediction, calculates the confidence score, sets the risk level and threat category, and writes advice that explains the risks of the site.

H. REPORT GENERATION AGENT

This agent puts the result into a report. The report lists the URL, predicted class, confidence, risk score, threat level and recommended actions.

V. SYSTEM IMPLEMENTATION

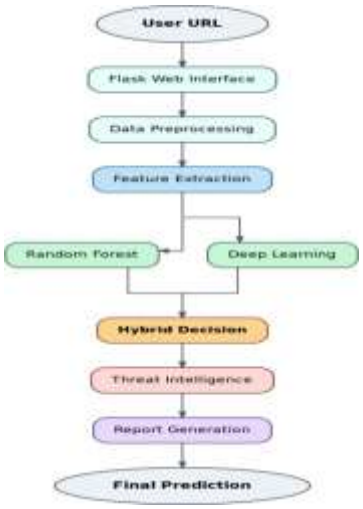


Fig. 2. System Implementation Workflow

Fig. 2 shows how the system is built. A URL entered in the Flask application is preprocessed and turned into features.

The Random Forest and the neural network then check it. The Hybrid Decision module combines their results, the threat analysis follows, and the report is shown to the user.

A. DATASET COLLECTION

We trained the models on a public malicious URL dataset from Kaggle [17]. Each URL in it is labelled Benign, Phishing, Malware or Defacement. We split the data into training and testing parts for building and checking the models.

B. DATA PREPROCESSING

We removed duplicate rows and missing values. The class labels were changed into numbers using label encoding. The records were shuffled so the order would not affect training. Finally, the data was split 80:20 into training and test set.Fig. 3 groups the 14 features by type.

FEATURE EXTRACTION

Each URL is described by lexical, structural and security-related features. These are turned into numbers and used by both models. The features are:

- URL length
- Domain length
- Number of dots
- Number of hyphens
- Number of digits
- Number of subdomains
- HTTPS usage
- Presence of an IP address
- Number of special characters
- Suspicious keywords
- Slash count
- Query parameters
- URL entropy
- Domain age (when available)

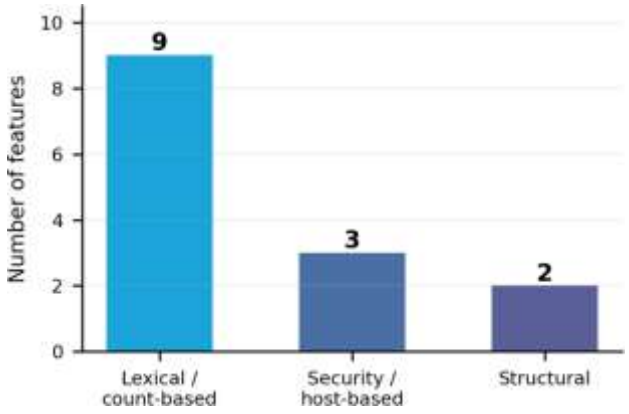


Fig. 3. Features by Type

C. MACHINE LEARNING IMPLEMENTATION

The Random Forest [18] builds many decision trees during training, and the class is chosen by majority vote. We chose it because it is accurate, stable, works well with many features and does not overfit easily.

D. DEEP LEARNING IMPLEMENTATION

The deep learning part is a 1D-CNN + BiLSTM model with an attention layer, built with TensorFlow [19]. It is trained on the processed feature vectors. The convolution layers pick up local patterns, the BiLSTM layer learns how the features relate to each other in both directions, and the attention layer gives more weight to the most useful features. Together they learn hidden patterns linked to harmful sites and can spot attacks that classic algorithms miss.

E. AGENTIC AI IMPLEMENTATION

The orchestrator runs the whole process. The validation agent checks the URL and the feature agent extracts its features. The two model agents predict separately. The threat intelligence agent works out the confidence and threat level, and the report agent writes the final output. Dividing the work like this improves automation, scalability and overall performance.

F. WEB APPLICATION IMPLEMENTATION

The front end is built with HTML, CSS and JavaScript, and the back end uses Flask. The user types a URL, and the page shows the predicted class, confidence score, risk score, threat level and recommendations. We also added Home, Login, Dashboard, About and Contact pages to make the system easier to use.

VI. EVALUATION

The proposed framework was tested on a public dataset using an 80:20 train-test split. Its performance was evaluated using four metrics: Accuracy, Precision, Recall, and F1-Score. The Random Forest model classified URLs based on their features, while the neural network identified complex patterns. The Hybrid Decision Agent combined the results of both models to make more reliable predictions.The framework achieved 98.92% accuracy, 98.75% precision, 98.1% recall, and 98.42% F1-score. These results show that the system can effectively identify four types of URLs: Benign, Phishing, Malware, and Defacement. By combining both models, the framework improves the reliability of threat detection.The agent-based system also automates feature extraction, URL prediction, threat analysis, and report generation, reducing manual effort. It can help improve online security by identifying potentially harmful URLs at an early stage. Its modular design makes it easier to update individual components when new threats emerge. Overall, the proposed framework shows promising results for real-time malicious URL detection, while further testing on different datasets can help evaluate its accuracy and reliability in real-world situations.

A. EXPERIMENTAL SETUP

The system was written in Python with Flask. The interface uses HTML, CSS and JavaScript, and the back end uses the Random Forest and neural network models. We ran the experiments on an Intel Core i5 processor with 8 GB RAM under Windows. Table I shows the setup.

TABLE I. EXPERIMENTAL SETUP

Parameter	Specification
Programming language	Python 3.x
Frontend	HTML, CSS, JavaScript
Backend framework	Flask
Machine learning model	Random Forest
Deep learning model	1D-CNN + BiLSTM + Attention
Dataset	Malicious URL dataset (Kaggle)
Operating system	Windows 10/11
IDE	Visual Studio Code

B. PERFORMANCE METRICS

We used Accuracy, Precision, Recall and F1-Score to measure performance. Fig. 4 plots the scores.

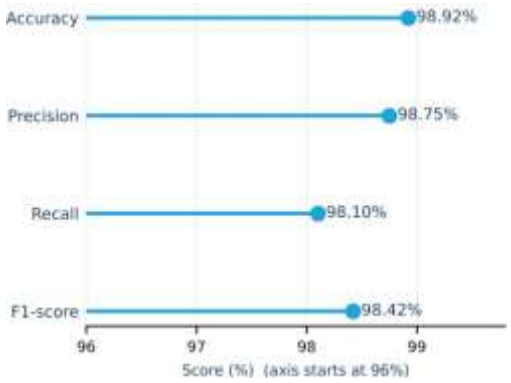


Fig. 4. Accuracy, Precision, Recall and F1-Score

C. EXPERIMENTAL RESULTS

We tested the web application with both safe and harmful URLs. It identified the different URL types correctly and showed the predicted class, confidence score, risk score and threat level on the Flask page. Fig. 5summarizes the 15 sample scans logged on the dashboard (Screenshot 3).

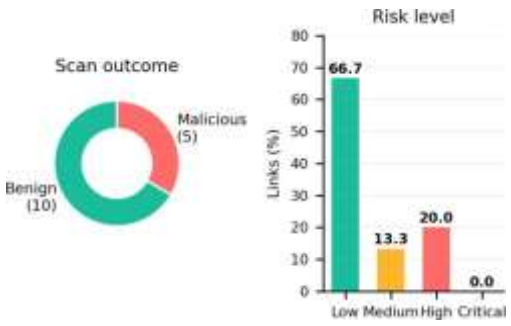
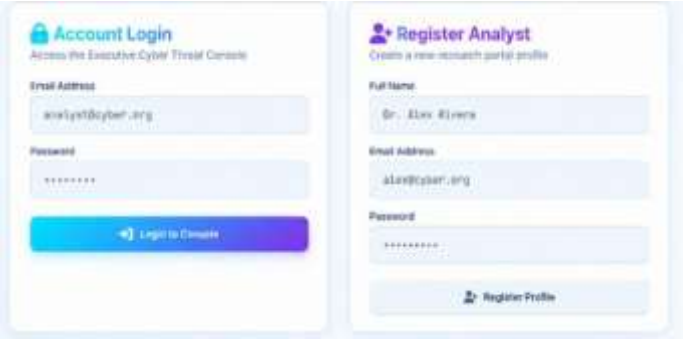


Fig. 5. Outcome and Risk Level of Sample Scans



Screenshot 1: **Login Page**

The Login Page lets registered users enter the system safely. New analysts can also register a profile here.



Screenshot 2: **Prediction / Threat Scanner Page**

On this page the user enters a URL and gets the predicted class, confidence score, risk level and threat analysis.



Screenshot 3: **Dashboard**

The Dashboard shows scan statistics, threat information and model performance numbers.

VII. DISCUSSION

The results show that the framework finds harmful URLs accurately and reliably. Joining Machine Learning, Deep Learning and Agentic AI works better than using a single technique.The Random Forest handles the lexical and security features well, while the neural network finds finer hidden patterns. When the Hybrid Decision Agent combines the two, both false positives and false negatives go down. The agent-based design also automates validation, feature extraction, prediction, analysis and reporting. This cuts manual work, helps the system scale and allows real-time use. The confidence score, risk level and advice help users judge a doubtful site before they open it.There are some limits. Accuracy depends on how good and varied the training data is, and new attack patterns will need fresh data and retraining from time to time. The neural network also needs more computing power than classic algorithms, so processing may take longer in large deployments. Overall, the framework is an efficient, scalable and smart way to detect harmful URLs and give web threat information. It gives dependable protection against phishing, malware and defacement attacks.

VIII. CONCLUSION AND FUTURE WORK

A. CONCLUSION

This paper presented a hybrid deep learning framework for adaptive malicious link detection and web threat intelligence. A Random Forest and a neural network study the URL features, and a hybrid decision step combines their predictions to label each URL as Benign, Phishing, Malware or Defacement. An agent-based design automates validation, feature extraction, threat analysis and reporting. Our tests showed high accuracy with fewer wrong predictions. The confidence score, risk level and advice help users decide before opening a doubtful link. The framework is therefore an efficient and scalable defence against growing online threats.

B. FUTURE WORK

In the future, the neural network can be trained on larger and more varied URL collections so it can handle new threats better. Transformer-based models and Graph Neural Networks may improve accuracy further. The agent framework can be extended with live threat intelligence feeds and cloud security services for continuous monitoring. Browser extensions, mobile apps and online deployment would bring real-time protection to everyday users, and explainable AI could make the predictions easier to understand.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering (Cyber Security) for its guidance and support during this project. They also thank the project supervisor, faculty members and the institution for their suggestions and encouragement. They also acknowledge the public malicious URL datasets and open-source tools used in building and testing the system.

REFERENCES

[1] A. Verma and P. Sharma, “Enhancing Online Safety: Phishing URL Detection Using Machine Learning and Explainable AI,” in Proc. 15th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2024.

[2] A. Nandu, J. Sosa, Y. Pant, Y. Panchal, and S. Sayyad, “Malicious URL Detection Using Machine Learning,” in Proc. 4th IEEE Asian Conf. Innovation in Technology (ASIANCON), 2024.

[3] R. B. Hani, M. Amoura, M. Ammourah, Y. A. Khalil, and M. Swailm, “Malicious URL Detection Using Machine Learning,” in Proc. 15th Int. Conf. Information and Communication Systems (ICICS), 2024.

[4] D. Orozco, L. Quesada, K. Ramirez-Benavides, and A. Lara, “Real-Time Malicious URL Detection,” in Proc. IEEE Latin-American Conf. Communications (LATINCOM), 2024.

[5] S. H. Shah, A. Garu, D. N. Nguyen, and M. Borowczak, “Feature Selection Framework for Optimizing ML-Based Malicious URL Detection,” in Proc. IEEE Cyber Awareness and Research Symposium (CARS), 2024.

[6] E. Nazeeruddin, G. Latif, and N. Mohammad, “Malicious URL Detection and Categorization Using Machine Learning Techniques,” in Proc. IEEE Int. Conf. Computational Intelligence and Communication Networks (CICN), 2024.

[7] P. Anusri, S. Sneha, P. Natarajan, T. Palanisamy, and S. K. Thangavel, “Shielding Cyberspace: A Machine Learning Approach for Malicious URL Detection,” in Proc. ICCCNT, 2024.

[8] A. K. Sharma, “DistilBERT Based Malicious URL Detection System,” in Proc. IEEE Int. Conf. Digital Society and Intelligent Systems (DSInS), 2024.

[9] J. Zhou, K. Zhang, A. Bilal, Y. Zhou, Y. Fan, W. Pan, X. Xie, and Q. Peng, “An Integrated CSPPC and BiLSTM Framework for Malicious URL Detection,” Scientific Reports, vol. 15, Art. no. 6659, 2025.

[10] H. Kibriya, R. Amin, S. S. Alshamrani, S. Rehman, M. Hassan, and F. S. Alsubaei, “Lightweight Malicious URL Detection Using Deep Learning and Large Language Models,” Scientific Reports, vol. 15, Art. no. 43044, 2025.

[11] Y. Tian, Y. Yu, J. Sun, and Y. Wang, “From Past to Present: A Survey of Malicious URL Detection Techniques, Datasets and Code Repositories,” Computer Science Review, vol. 58, Art. no. 100810, 2025.

[12] S. Kailas and R. Roopalakshmi, “Think Before You Click—Malicious URL Detection in Cybersecurity: A Systematic Review and Research Roadmap,” IEEE Access, vol. 13, pp. 154305–154325, 2025.

[13] A. Kulkarni, V. Balachandran, D. M. Divakaran, and T. Das, “From ML to LLM: Evaluating the Robustness of Phishing Webpage Detection Models Against Adversarial Attacks,” arXiv:2407.20361, 2024.

[14] S. Gopali, A. S. Namin, F. Abri, and K. S. Jones, “The Performance of Sequential Deep Learning Models in Detecting Phishing Websites Using Contextual Features of URLs,” arXiv:2404.09802, 2024.

[15] I. Schvartzman, R. Sarussi, M. Ashkenazi, I. Kringel, Y. Tocker, and T. F. Shohet, “A New Dataset and Methodology for Malicious URL Classification,” arXiv:2501.00356, 2025.

[16] L. Chen and L. Meng, “Metadata Driven Malicious URL Detection Using RoBERTa-Large and Multi-Source Network Threat Intelligence,” Scientific Reports, vol. 16, Art. no. 5449, 2026.

[17] Kaggle, “Malicious URLs Dataset,” 2025. [Online]. Available: https://www.kaggle.com/

[18] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” 2025. [Online]. Available: https://scikit-learn.org/

[19] TensorFlow Developers, “TensorFlow: An End-to-End Open Source Machine Learning Platform,” 2025. [Online]. Available: https://www.tensorflow.org/